

Trabajo Práctico 6

Inciso 1

¿Cuál es el puerto por defecto que se utiliza en los siguientes servicios?

Web / SSH / DNS / Web Seguro / POP3 / IMAP / SMTP

Investigue en qué lugar en Linux y en Windows está descrita la asociación utilizada por defecto para cada servicio

Respuesta:

Servicio	Protocolo	Puerto
Web	TCP	80
SSH	TCP	22
DNS	UDP/TCP	53
Web seguro	TCP (UDP en HTTP/3 con quic)	443
POP3	TCP	110
IMAP	TCP	143
SMTP	TCP	25 (587 para submission/msa)

- **En Linux:** La asociación estandarizada entre los nombres de los servicios, sus números de puerto y el protocolo de transporte correspondiente (TCP/UDP) está descrita en el archivo **/etc/services**. Herramientas de red como `ss` o `netstat` leen este archivo para traducir los números de puerto a nombres de servicios legibles. Se puede consultar o filtrar mediante órdenes como `grep` (por ejemplo: `grep domain /etc/services`).
- **En Windows:** La tabla de correspondencias equivalente se encuentra ubicada en el archivo de texto del sistema **C:\Windows\System32\drivers\etc\services** (el cual se puede

inspeccionar con cualquier editor de texto como el Bloc de notas).

```
File Edit View Terminal Tools Help
redes@debian:~$ cat /etc/services
# Network services, Internet style
#
# Updated from https://www.iana.org/assignments/service-names-port-num
bers/service-names-port-numbers.xhtml .
#
# New ports will be added on request if they have been officially assi
gned
# by IANA and used in the real-world or are needed by a debian package
.
# If you need a huge list of used numbers please install the nmap pack
age.

tcpmux          1/tcp                      # TCP port service mul
tiplexer
echo            7/tcp
echo            7/udp
discard         9/tcp          sink null
discard         9/udp          sink null
sysstat         11/tcp         users
daytime         13/tcp
daytime         13/udp
netstat         15/tcp
```

Inciso 2

Investigue qué es multicast. ¿Sobre cuál de los protocolos de capa de transporte

funciona? ¿Se podría adaptar para que funcione sobre el otro protocolo de capa de

transporte? ¿Por qué?

[Multicast cisco](#)

Respuesta

Multicast es un esquema de comunicación de red en el cual un único emisor transmite paquetes de datos a un **grupo de receptores interesados** de forma simultánea a través de una única operación de envío.

A diferencia de:

- **Unicast (Unidifusión):** Comunicación de un emisor a un único receptor (punto a punto).
- **Broadcast (Difusión):** Comunicación de un emisor a *todos* los hosts de la red local.

En **multicast**, los paquetes se envían a una **dirección IP de grupo de multidifusión** (en IPv4, direcciones de Clase D en el rango 224.0.0.0 a 239.255.255.255). La infraestructura de red (routers y switches) se encarga de replicar los paquetes únicamente en los enlaces que conducen a hosts que se hayan unido explícitamente a dicho grupo.

Este mismo funciona sobre el protocolo de **UDP**. Al ser este sin conexión y no fiable, UDP empaqueta los datos en datagramas y los entrega a la capa de red con la dirección multicast destino.

¿Se podría adaptar para que funcione sobre otro? Ejemplo TCP

La respuesta es no, porque:

- **TCP es estrictamente punto a punto (orientado a conexión):** Una conexión TCP existe de manera exclusiva entre un único emisor y un único receptor
- **Demultiplexación por 4-upla:** Un socket de conexión TCP en el sistema operativo queda determinado unívocamente por una **4-upla**: (IP origen, puerto origen, IP destino, puerto destino) . Esto exige un estado de sesión individualizado por cada par cliente-servidor, lo cual es incompatible conceptualmente con enviar un solo segmento hacia un grupo indeterminado de receptores.
- **Implosión de reconocimientos (ACK Storm / ACK Implosion):** TCP garantiza la entrega fiable mediante reconocimientos (ACKs) explícitos/acumulativos y temporizadores de retransmisión. Si un emisor intentara enviar tráfico TCP en multicast a cientos o miles de receptores simultáneamente, recibiría de golpe una gran cantidad de ACKs que colapsaría tanto al servidor emisor como a los enlaces de red.
- **Imposibilidad de gestionar el control de flujo y de congestión:** TCP ajusta dinámicamente su velocidad de transmisión según la capacidad del receptor (**ventana de recepción**) y el estado de la red (**ventana de congestión**). En un grupo multicast, cada receptor tiene diferentes capacidades de buffer y atraviesa enlaces con distintos niveles de congestión o retardo; por ende, no existe una única tasa de envío o tamaño de ventana válido para todo el grupo

Inciso 3

Investigue cómo funciona el protocolo de aplicación FTP teniendo en cuenta las diferencias en su funcionamiento cuando se utiliza el modo activo de cuando se utiliza el modo pasivo ¿En qué se diferencian estos tipos de comunicaciones del resto de los protocolos de aplicación vistos?

Respuesta:

El protocolo de aplicación **FTP (File Transfer Protocol)**, definido en la RFC 959, se utiliza para la transferencia de archivos entre un cliente y un servidor a través del protocolo de transporte **TCP**.

A diferencia de la mayoría de los protocolos de aplicación, FTP se caracteriza por utilizar **dos canales de conexión TCP independientes** (arquitectura fuera de banda o *out-of-band*):

1. **Canal de control (Puerto TCP 21 por defecto):** Se establece al inicio de la sesión y permanece abierto durante toda la interacción para enviar comandos (autenticación, listado de directorios, comandos de transferencia) y recibir códigos de respuesta.
2. **Canal de datos:** Se abre de manera dinámica únicamente cuando se solicita transferir un archivo o listar un directorio, y se cierra al finalizar dicha transferencia.

La forma en que se establece el canal de datos determina si se opera en **Modo Activo** o **Modo Pasivo**.

Modo activo:

En el modo activo, el establecimiento del canal de datos es **iniciado por el servidor** hacia el cliente:

1. **Conexión de Control:** El cliente abre una conexión TCP desde un puerto efímero local N hacia el puerto **TCP 21** del servidor.
2. **Solicitud de Datos:** Cuando el cliente desea transferir un archivo, abre un segundo puerto efímero local M en su máquina y envía al servidor el comando `PORT IP_cliente,puerto_M` a través del canal de control.
3. **Conexión de Datos:** El servidor recibe el comando e inicia una nueva conexión TCP desde su puerto reservado TCP 20 hacia la dirección IP del cliente y su puerto M .

4. **Inconveniente:** Si el cliente se encuentra detrás de un firewall o un router con traducción de direcciones (NAT), la solicitud de conexión entrante iniciada por el servidor hacia el puerto M del cliente será rechazada o bloqueada por seguridad.

Modo pasivo:

El modo pasivo fue diseñado para solucionar los problemas de conectividad del modo activo, haciendo que **ambas conexiones sean iniciadas por el cliente**:

1. **Conexión de Control:** El cliente se conecta desde un puerto efímero N al puerto **TCP 21** del servidor.
2. **Solicitud de Modo Pasivo:** Cuando se requiere transferir datos, el cliente envía el comando `PASV` sobre el canal de control.
3. **Respuesta del Servidor:** El servidor abre un puerto efímero aleatorio P en su propia máquina y le responde al cliente con el mensaje `227 Entering Passive Mode (IP_servidor,puerto_P)`.
4. **Conexión de Datos:** El cliente **inicia la conexión TCP** desde su puerto efímero $N + 1$ hacia la IP del servidor y el puerto efímero P que este le indicó.
5. **Ventaja:** Dado que ambas conexiones TCP se originan desde el cliente hacia el servidor, los cortafuegos y routers NAT del lado del cliente permiten el tráfico saliente sin inconvenientes.

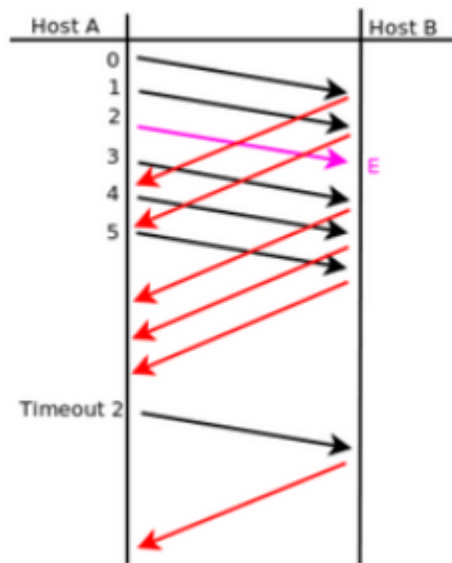
El funcionamiento de FTP se diferencia sustancialmente de otros protocolos de aplicación vistos (como HTTP, SMTP, POP3, IMAP, SSH o DNS) en dos aspectos fundamentales:

- **Separación en dos canales (Control fuera de banda / Out-of-band):**
 - Protocolos como **HTTP**, **SMTP** o **SSH** utilizan una **única conexión TCP** por la cual se transmiten de forma entrelazada (*in-band*) tanto las peticiones de control como el contenido de los datos.
 - **FTP**, en cambio, mantiene el canal de control separado del canal de transporte de datos. Los archivos nunca viajan por el puerto 21.
- **Dirección de inicio de la conexión TCP:**

- En protocolos convencionales cliente-servidor (HTTP, HTTPS, SSH, DNS), **el cliente es siempre quien inicia todas las conexiones TCP** hacia los puertos bien conocidos del servidor.
- En **FTP Modo Activo**, se invierte este rol para el canal de datos: **el servidor es quien inicia la conexión de retorno** hacia el puerto efímero expuesto por el cliente.

Inciso 4

Suponiendo Selective Repeat; tamaño de ventana 4 y sabiendo que E indica que el mensaje llegó con errores. Indique en el siguiente gráfico, la numeración de los ACK que el host B envía al Host A.



[Selective-repeat apunte](#)
[ACK selective repeat](#)

Respuesta:

Siguiendo las flechas negras (paquetes de datos enviados por A) y las rojas (ACKs enviados por B de abajo hacia arriba), la secuencia de ACKs que el Host B emite es la siguiente:

1. **Llega el Paquete 0 (bien):** Host B responde enviando **ACK 0**.
2. **Llega el Paquete 1 (bien):** Host B responde enviando **ACK 1**.
3. **Llega el Paquete 2 con error (E):** Host B lo descarta → **No se envía ACK** para el 2 en este momento.
4. **Llega el Paquete 3 (bien, pero desordenado):** Como está dentro de la ventana de recepción ($N = 4$), B lo guarda en el búfer y responde

enviando **ACK 3**.

5. **Llega el Paquete 4 (bien, desordenado):** B lo guarda en el búfer y responde enviando **ACK 4**.
6. **Llega el Paquete 5 (bien, desordenado):** B lo guarda en el búfer y responde enviando **ACK 5**.
7. **Timeout del Paquete 2:** El Host A, al no recibir el ACK de 2, retransmite únicamente el paquete 2.
8. **Llega el Paquete 2 retransmitido (bien):** El Host B lo recibe, lo junta con los paquetes 3, 4 y 5 que tenía en el búfer, entrega todo el bloque ordenado a la capa superior y responde enviando **ACK 2**.

Inciso 5

¿Qué restricción existe sobre el tamaño de ventanas en el protocolo Selective Repeat?

Respuesta:

En el protocolo **Repetición Selectiva** (*Selective Repeat* - SR), la restricción fundamental es que el tamaño de la ventana (N) debe ser menor o igual a la mitad del tamaño del espacio de números de secuencia disponible:

$$\text{Tamaño de la ventana } (N) \leq \frac{\text{Tamaño del espacio de números de secuencia}}{2}$$

Si los números de secuencia se representan mediante un campo de k bits en la cabecera (lo que genera un espacio total de 2^k números de secuencia), la restricción se expresa formalmente como:

$$N \leq 2^{k-1}$$

¿Por qué existe esta restricción?

A diferencia de protocolos como *Go-Back-N*, en Repetición Selectiva el emisor y el receptor no siempre tienen una visión idéntica de qué paquetes se han entregado correctamente, por lo que sus ventanas no avanzan exactamente al mismo tiempo y el receptor almacena en un *buffer* los paquetes fuera de orden.

Si el tamaño de la ventana supera la mitad del espacio de números de secuencia, surge una ambigüedad insalvable para el receptor, que no puede distinguir si un paquete entrante es una **retransmisión de un paquete antiguo** o la **primera llegada de un paquete nuevo**:

1. **Escenario A (Pérdida de reconocimientos):** El emisor transmite los paquetes de la ventana actual y el receptor los acepta enviando sus respectivos ACKs; sin embargo, todos los ACKs se pierden en el canal. Al vencer el temporizador, el emisor retransmite el primer paquete de la ventana.
2. **Escenario B (Transmisión y avance exitoso):** Los paquetes y sus ACKs llegan correctamente, por lo que la ventana del receptor avanza. El emisor transmite los siguientes paquetes del flujo, reutilizando los números de secuencia iniciales del ciclo previo.

Si la ventana es demasiado grande ($N > 2^{k-1}$), el receptor observa llegar un paquete con el mismo número de secuencia en ambos escenarios y no tiene forma de saber si debe descartarlo por ser duplicado o entregarlo a la capa superior como datos nuevos.

Inciso 6

De acuerdo a la captura TCP de la siguiente figura, indique los valores de los campos borroneados.

No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000	172.20.1.1	172.20.1.100	TCP	74	41749 > vce [] Seq= Win=5840 Len=0 MSS=1460 SACK_PERM=1 TSval=270132 TSecr=0
2	0.001264	172.20.1.100	172.20.1.1	TCP	74	vce > 41749 [SYN, ACK] Seq=1047471501 Ack=3933822138 Win=5792 Len=0 MSS=1460 SACK_PERM=1
3	0.001341			TCP	66	> [] Seq= Ack= Win=5888 Len=0 TSval=270132 TSecr=1877442

Internet Protocol Version 4, Src: 172.20.1.100 (172.20.1.100), Dst: 172.20.1.1 (172.20.1.1)

Transmission Control Protocol, Src Port: vce (11111), Dst Port: 41749 (41749), Seq: 1047471501, Ack: 3933822138, Len: 0

Source port: vce (11111)
Destination port: 41749 (41749)
[Stream index: 0]
Sequence number: 1047471501
Acknowledgement number: 3933822138
Header length: 40 bytes

Flags: 0x012 (SYN, ACK)

000. = Reserved: Not set
...0 = Nonce: Not set
.... 0... = Congestion Window Reduced (CWR): Not set
.... .0.. = ECN-Echo: Not set
.... ..0. = Urgent: Not set
.... ...1 = Acknowledgement: Set
....0.. = Push: Not set
....0.. = Reset: Not set
... ..1. = Syn: Set
....0 = Fin: Not set
Window size value: 5792
[Calculated window size: 5792]
Checksum: 0x9803 [validation disabled]

[Three way handshake tcp](#)

Respuesta:

- **Paquete 1 (Cliente → Servidor):**
 - **Tipo:** Segmento **SYN** (Inicio de conexión por parte del cliente).
 - **Seq:** (Valor borroneado del cliente, por ejemplo, supongamos X o un número base).
- **Paquete 2 (Servidor → Cliente):**
 - **Tipo:** Segmento **SYN, ACK**.
 - **Seq:** Corresponde al número de secuencia inicial elegido por el servidor (ej. 1047471501).
 - **Ack:** Como confirma el SYN del cliente, su valor es el **Seq del cliente + 1** (es decir, $X + 1$).
- **Paquete 3 (Cliente → Servidor):**
 - **Tipo:** Segmento **ACK** (Tercer paso del handshake).
 - **Seq:** Debe continuar el flujo del cliente, por lo que su valor es el Seq inicial del cliente más los datos enviados en el primer segmento (como el SYN consume 1 número virtual de secuencia, el Seq pasa a ser $X + 1$).
 - **Ack:** Como confirma el SYN-ACK del servidor, el valor del Ack es el **Seq del servidor + 1** (es decir, $1047471501 + 1 = 1047471502$).

Inciso 7

Dada la sesión TCP de la figura, completar los valores marcados con un signo de interrogación.

Time	10.0.0.10	10.0.1.10	Comment
1.360	(54762) →	SYN → (10000)	Seq = 0
1.360	(54762) ←	SYN, ACK → (10000)	Seq = 0 Ack = 1
1.360	(54762) →	ACK → (10000)	Seq = ? Ack = ?
3.581	(54762) →	PSH, ACK - Len: 7 → (10000)	Seq = 1 Ack = 1
3.581	(54762) ←	ACK → (10000)	Seq = 1 Ack = ?
8.796	(54762) →	PSH, ACK - Len: 9 → (10000)	Seq = 8 Ack = 1
8.797	(54762) ←	ACK → (10000)	Seq = 1 Ack = ?
14.382	(54762) →	PSH, ACK - Len: 5 → (10000)	Seq = 17 Ack = 1
14.382	(54762) ←	ACK → (10000)	Seq = 1 Ack = ?
15.190	(54762) →	FIN, ACK → (10000)	Seq = ? Ack = 1
15.190	(54762) ←	FIN, ACK → (10000)	Seq = 1 Ack = ?
15.190	(54762) →	ACK → (10000)	Seq = ? Ack = 2

Respuesta

Time	10.0.0.10	10.0.1.10	Comment
1.360	(54762) →	SYN → (10000)	Seq = 0
1.360	(54762) ←	SYN, ACK → (10000)	Seq = 0 Ack = 1
1.360	(54762) →	ACK → (10000)	Seq = ? Ack = ? 1,1
3.581	(54762) →	PSH, ACK - Len: 7 → (10000)	Seq = 1 Ack = 1
3.581	(54762) ←	ACK → (10000)	Seq = 1 Ack = ? 1,8
8.796	(54762) →	PSH, ACK - Len: 9 → (10000)	Seq = 8 Ack = 1
8.797	(54762) ←	ACK → (10000)	Seq = 1 Ack = ? 1,17
14.382	(54762) →	PSH, ACK - Len: 5 → (10000)	Seq = 17 Ack = 1
14.382	(54762) ←	ACK → (10000)	Seq = 1 Ack = ? 1,22
15.190	(54762) →	FIN, ACK → (10000)	Seq = ? Ack = 1 22,1
15.190	(54762) ←	FIN, ACK → (10000)	Seq = 1 Ack = ? 1,23
15.190	(54762) →	ACK → (10000)	Seq = ? Ack = 2 22,2

✍ Nota de ayuda

Siguiente Seq = Seq actual + Len (longitud de datos del segmento)

Ack a enviar = Seq recibido + Len del segmento recibido